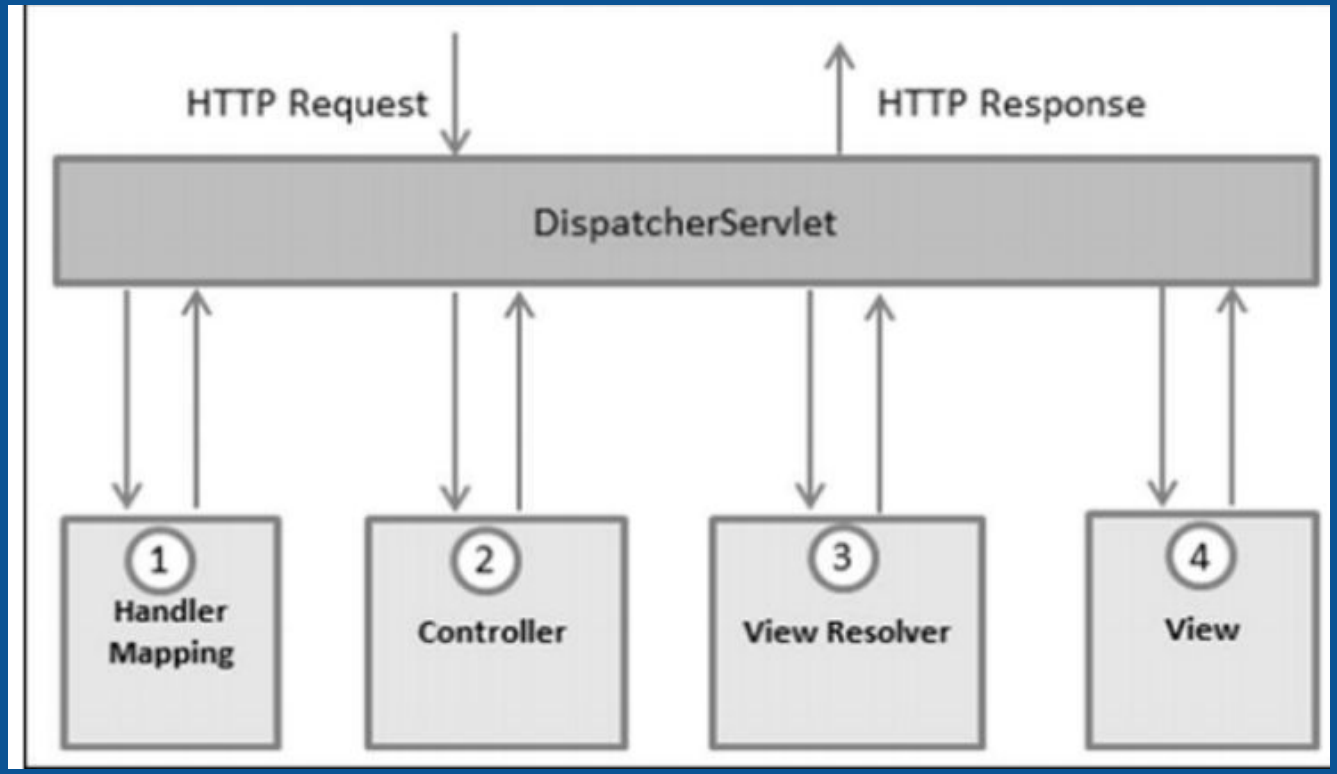
Spring Boot para Eclipse

...

MVC

El marco Spring Web MVC proporciona una arquitectura de modelo-vista-controlador y componentes listos que se pueden usar para desarrollar aplicaciones web flexibles y de acoplamiento flexible. El patrón MVC da como resultado la separación de los diferentes aspectos de la aplicación (lógica de entrada, lógica de negocios y lógica de interfaz de usuario), al tiempo que proporciona un acoplamiento flexible entre estos elementos.

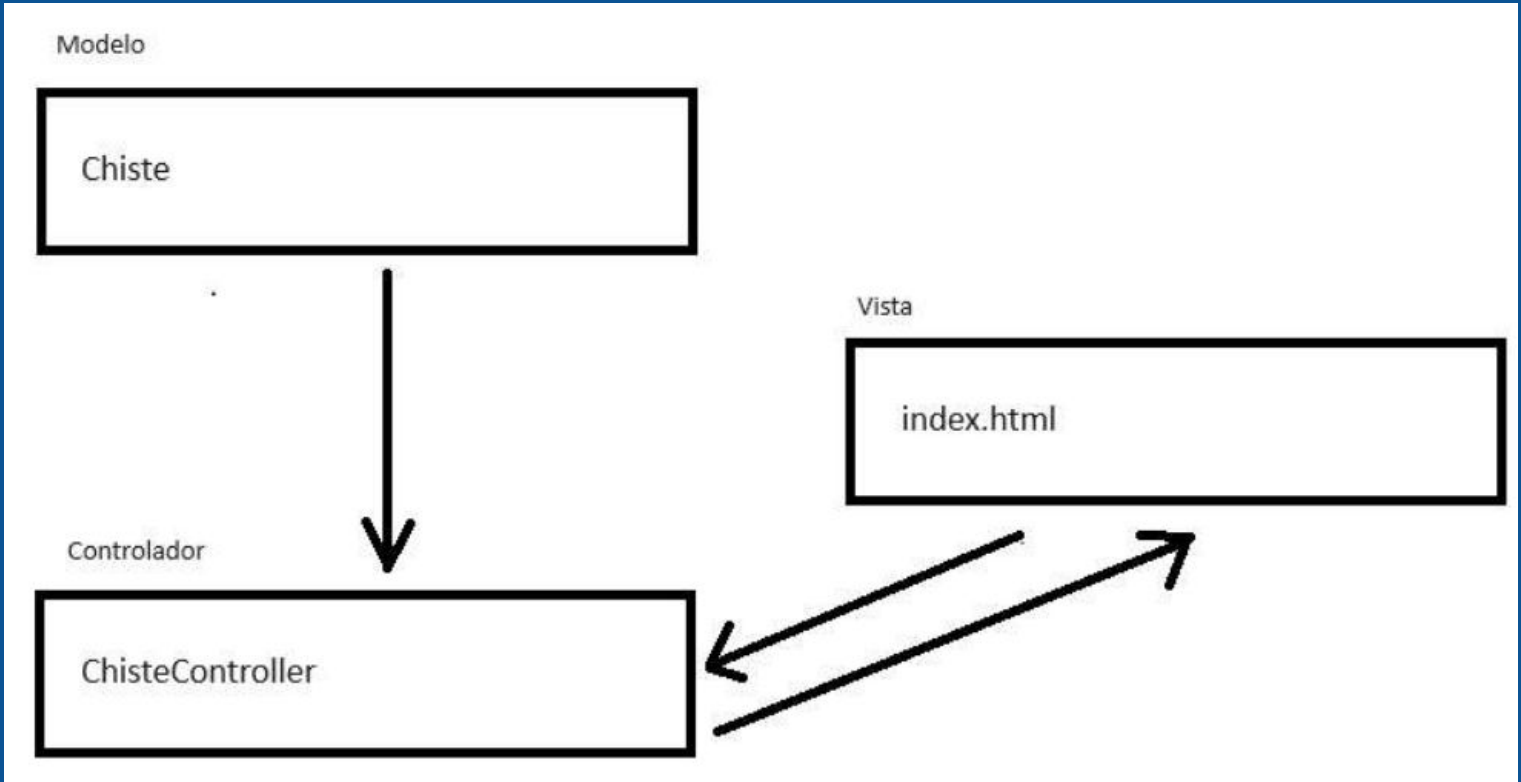
MVC


HTTP Request
HTTP Response
DispatcherServlet
1
Handler Mapping
2
Controller
3
View Resolver
4
View

COMO FUNCIONA :

- Después de recibir una solicitud HTTP, DispatcherServlet consulta el **HandlerMapping** para llamar al Controller adecuado.
- El controlador toma la solicitud y llama a los métodos de servicio apropiados en función del método **GET** o **POST** utilizado. El método de servicio establecerá los datos del modelo en función de la lógica de negocios definida y devuelve el nombre de la vista a DispatcherServlet.
- DispatcherServlet tomará la ayuda de **ViewResolver** para recoger la vista definida para la solicitud.
- Una vez finalizada la vista, DispatcherServlet pasa los datos del modelo a la vista, que finalmente se representa, en los navegadores.

COMO FUNCIONA EN MI PROYECTO:

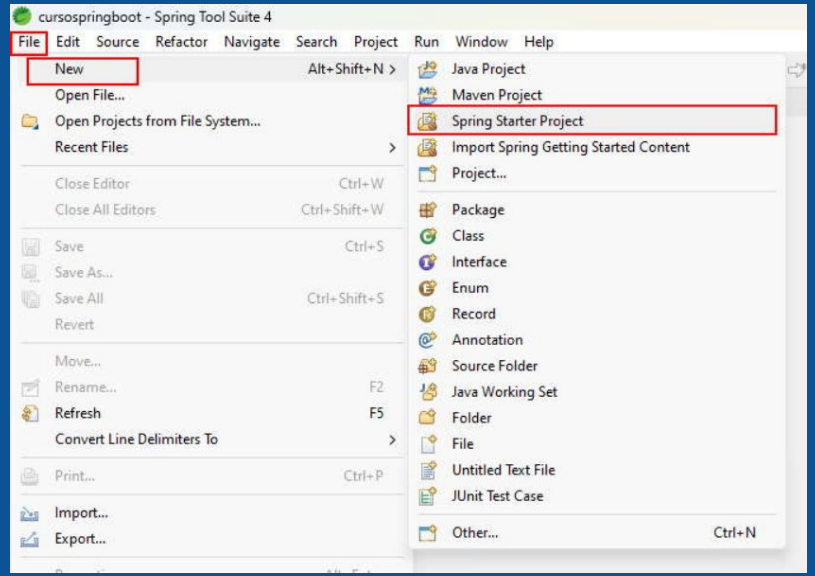


spring boot

Java

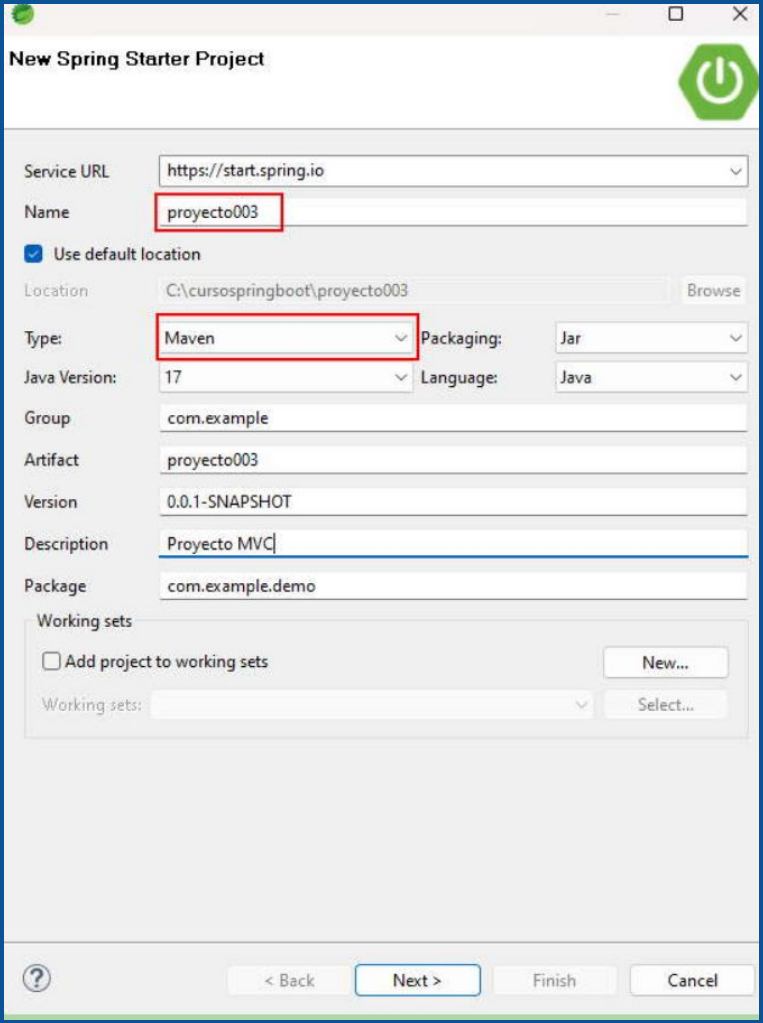
Los pasos para crear el proyecto:

- Desde el menú de opciones seleccionamos File-> New -> Spring Starter Project:



spring boot

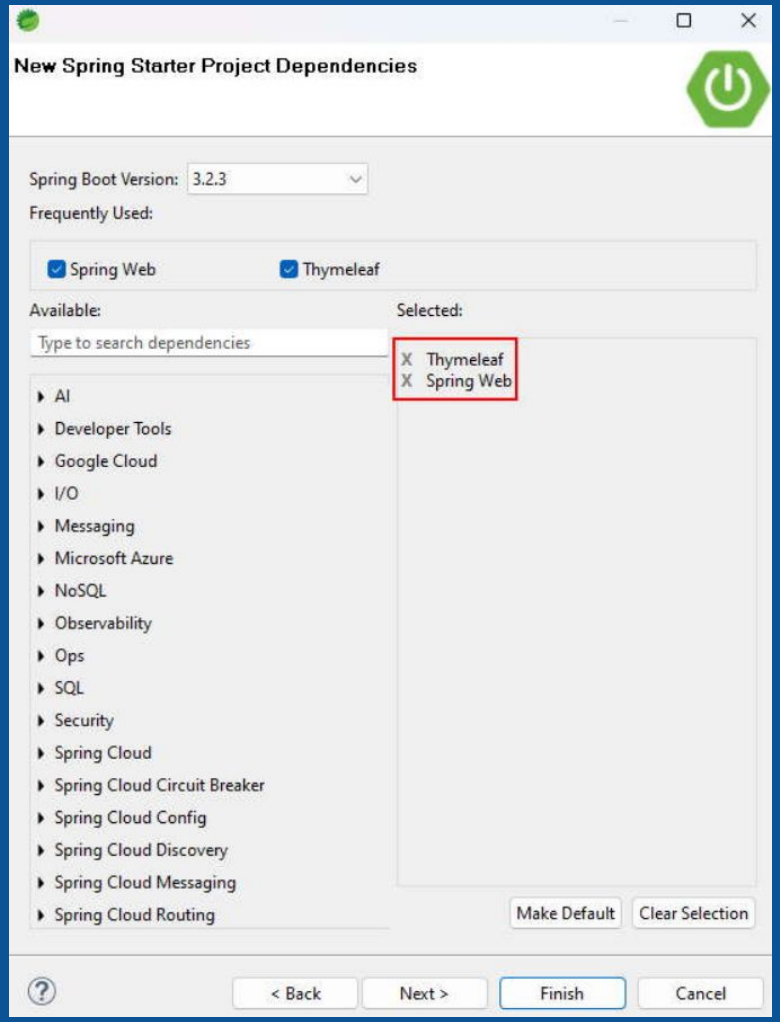
- En el primer diálogo procedemos a definir el nombre de nuestro proyecto. Utilizaremos Maven como herramienta de gestión de proyecto.



Java

spring boot

- En el segundo diálogo seleccionamos las dependencias "Spring Web" y "Thymeleaf"



Java

Ahora en el paquete 'com.example.demo' procedemos a crear la carpeta 'model' presionando el botón derecho del mouse:

cursospringboot - Spring Tool Suite 4

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- proyecto001 [boot]
- proyecto002 [boot]
- proyecto003 [boot]
  - src/main/java
    - com.example.demo
  - src/main/resources
  - src/test/java
  - JRE System Library [Ja
  - Maven Dependencies
  - src
  - target
  - HELP.md
  - mvnw
  - mvnw.cmd
  - pom.xml

Boot Dashboard

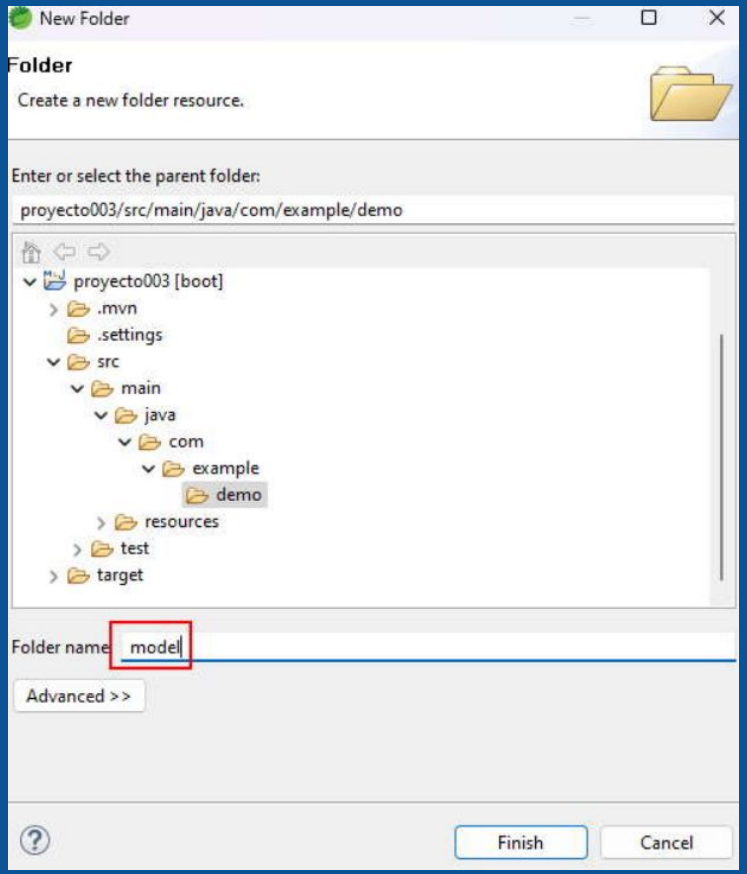
Type tags, projects, or working

New	>
Go Into	
Open in New Window	
Open Type Hierarchy	F4
Show In	Alt+Shift+W >
Show in Local Terminal	>
Copy	Ctrl+C
Copy Qualified Name	
Paste	Ctrl+V
Delete	Delete
Build Path	>
Source	Alt+Shift+S >
Refactor	Alt+Shift+T >
Import...	
Export...	
Source	>
References	>
Declarations	>
Refresh	F5

Java Project	
Maven Project	
Spring Starter Project	
Import Spring Getting Started Content	
Project...	
Package	
Class	
Interface	
Enum	
Record	
Annotation	
Source Folder	
Java Working Set	
Folder	
File	
Untitled Text File	
JUnit Test Case	
Other...	Ctrl+N

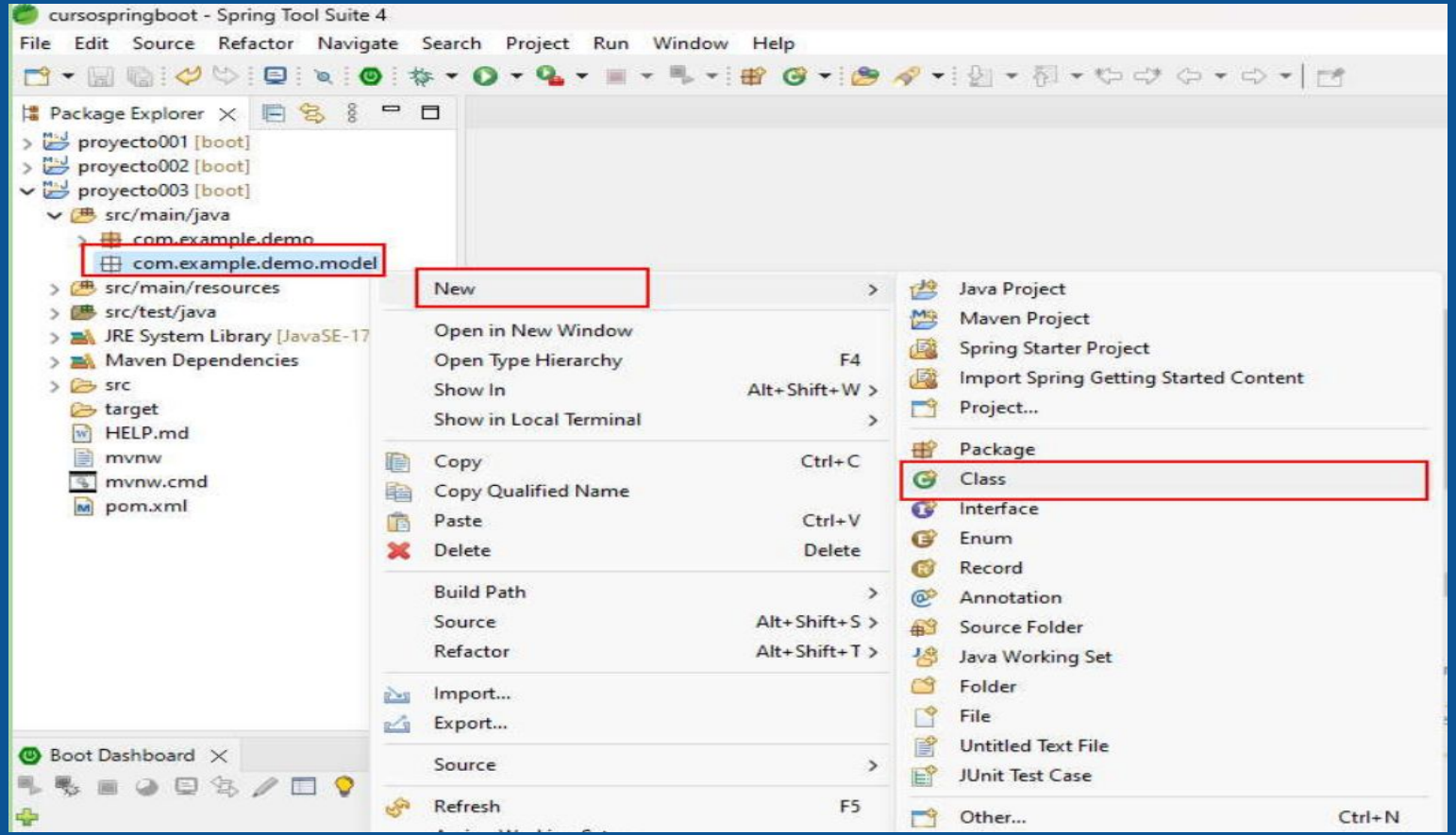
spring boot

- Y en el diálogo procedemos a indicar el nombre 'model' (es una convención comúnmente adoptada por muchos desarrolladores y equipos debido a su claridad y facilidad de comprensión):

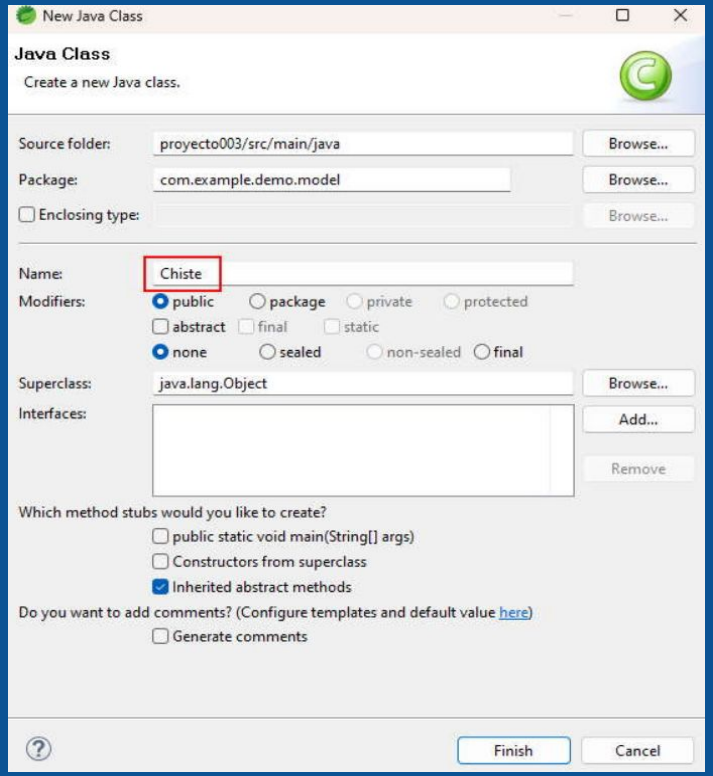


Java

En la carpeta model procedemos a crear la clase Chiste:



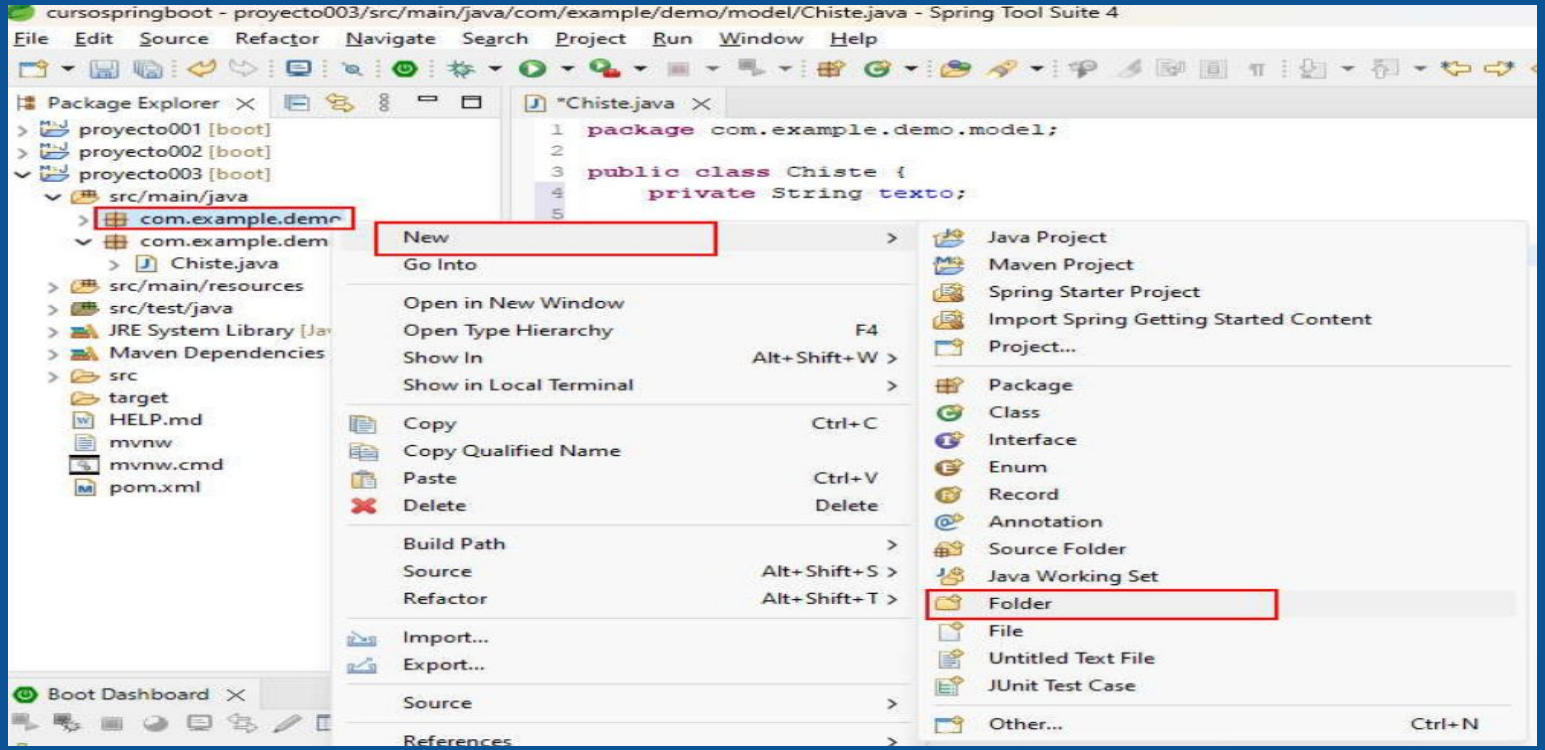
Y en el diálogo procedemos a indicar el nombre Chiste:



El código de la clase Chiste es el siguiente:

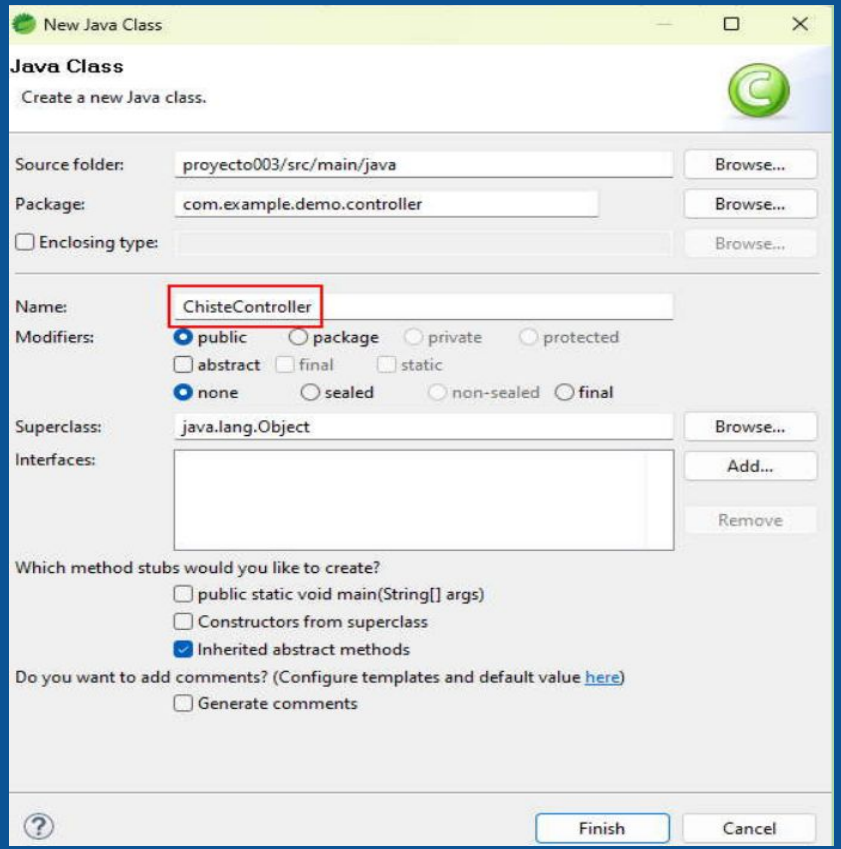
```
package com.example.demo.model;

public class Chiste {
    private String texto;

    public Chiste(String texto) {
        this.texto = texto;
    }

    public String getTexto() {
        return texto;
    }

    public void setTexto(String texto) {
        this.texto = texto;
    }
}
```

Ahora nuevamente en el paquete 'com.example.demo' procedemos a crear la carpeta 'controller' presionando el botón derecho del mouse:



spring boot

Y en el diálogo procedemos a indicar el nombre ChisteController:



Java

El código de la clase ChisteController en la siguiente pagina:

```java
package com.example.demo.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import com.example.demo.model.Chiste;

@Controller
public class ChisteController {

    @GetMapping("/")
    public String inicio(Model model) {
        List<Chiste> chistes = new ArrayList<>();
        chistes.add(new Chiste("¿Qué le dice un 0 a un 8? Bonito cinturón."));
        chistes.add(new Chiste("¿Qué hace una abeja en el gimnasio? ¡Zum-ba!"));
        chistes.add(new Chiste("¿Cuál es el colmo de un electricista? Que su hijo sea un
        chistes.add(new Chiste("¿Por qué estás hablando con la pared? ¡Porque la mesa no

        Chiste chisteAzar = chistes.get((int)(Math.random() * chistes.size()));

        model.addAttribute("chiste", chisteAzar);
        return "index";
    }
}
```

La anotación @Controller indica que esta clase es un controlador de Spring MVC. Esto significa que puede manejar solicitudes HTTP entrantes.

```
@GetMapping("/")
public String inicio(Model model) {
```

El método 'inicio' maneja solicitudes HTTP GET a la raíz del contexto de la aplicación ("/"). Es decir, cuando alguien accede a la página principal de la aplicación, este método se ejecuta

spring boot

Java

Toma un objeto Model como parámetro, que es utilizado para pasar datos a la vista.

```
List<Chiste> chistes = new ArrayList<>();
chistes.add(new Chiste("¿Qué le dice un 0 a un 8? Bonito cinturón."));
chistes.add(new Chiste("¿Qué hace una abeja en el gimnasio? ¡Zum-ba!"));
chistes.add(new Chiste("¿Cuál es el colmo de un electricista? Que su hijo sea un apagado."));
chistes.add(new Chiste("¿Por qué estás hablando con la pared? ¡Porque la mesa no me responde!"));

Chiste chisteAzar = chistes.get((int)(Math.random() * chistes.size()));
```

spring boot

Java

En este bloque se crea una lista de chistes (chistes) y se agregan instancias de la clase Chiste a esta lista.

Luego, se selecciona un chiste aleatorio de la lista usando un índice generado aleatoriamente.

```
model.addAttribute("chiste", chisteAzar);
return "index";
```

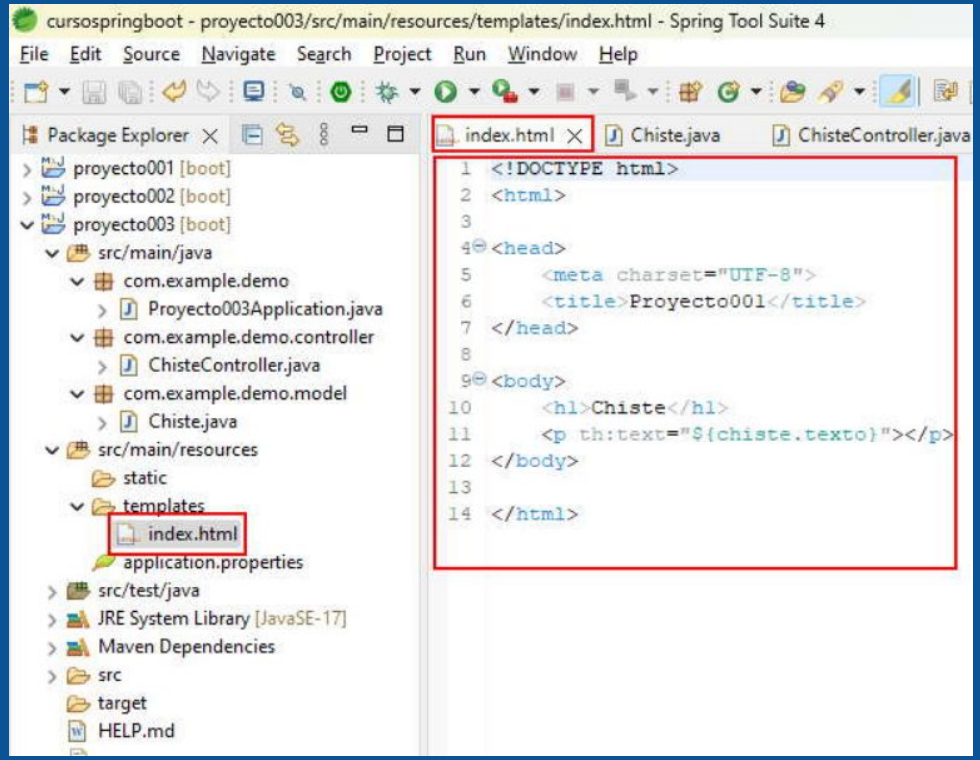
spring boot

Java

Se agrega el chiste aleatorio al objeto Model con el nombre "chiste".

Se devuelve el nombre de la vista que se mostrará al usuario. En este caso, la vista se llama "index".

Ahora procedamos a crear la vista en la carpeta 'template' el archivo index.html:



spring boot

Java

El código fuente de la página index.html es:

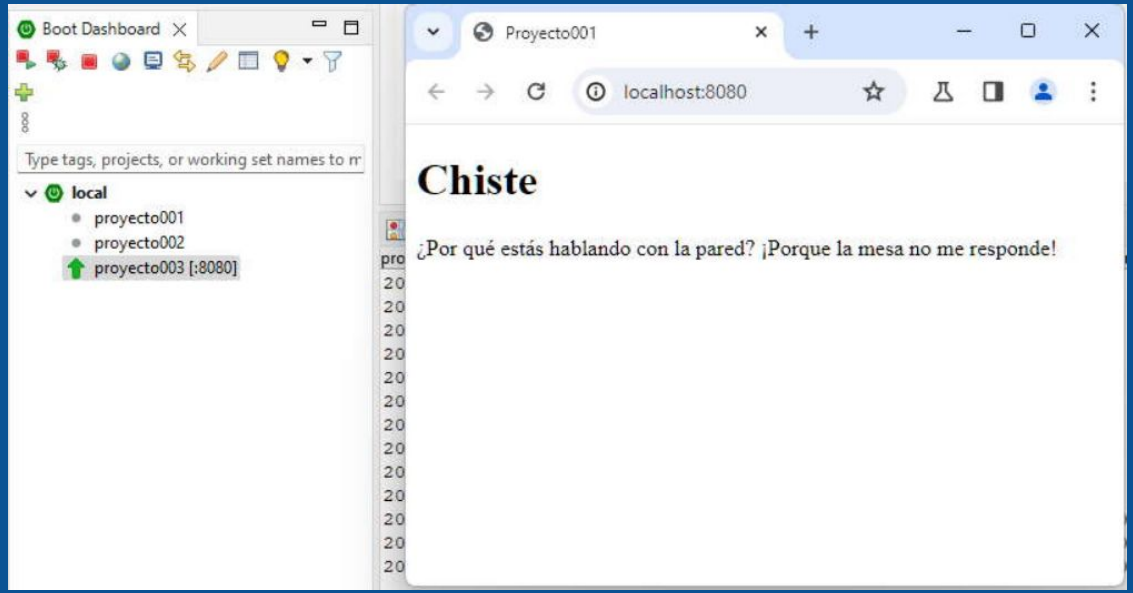
```
<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8">
    <title>Proyecto003</title>
</head>

<body>
    <h1>Chiste</h1>
    <p th:text="${chiste.texto}"></p>
</body>

</html>
```

spring boot

Java

th:text="${chiste.texto}": La expresión ${chiste.texto} se evaluará y el texto resultante se mostrará en el párrafo. chiste.texto se refiere al atributo "texto" del objeto "chiste" que se pasó desde el controlador a la vista.

Ahora si ejecutamos la aplicación (recordar siempre de guardar todos los archivos modificados y volver a reiniciar el servidor) vemos que nos muestra chistes distintos cada vez que actualizamos la página web:



spring boot

Java

Paso	Descripción
1	Cree un proyecto con un nombre HelloWeb en un paquete com.tutorialspoint como se explica en el capítulo Spring MVC - Hello World.
2	Cree las clases Java User, UserController en com.tutorialspointpackage.
3	Cree archivos de vista user.jsp, users.jsp en la subcarpeta jsp.
4	El paso final es crear el contenido de los archivos de origen y configuración y exportar la aplicación como se explica a continuación.

spring boot

1 Crear :
1 parte

User.java

```
package com.tutorialspoint;

public class User {

   private String username;
   private String password;
   private String address;

   public String getUsername() {
      return username;
   }
   public void setUsername(String username) {
      this.username = username;
   }

   public String getPassword() {
      return password;
   }
```

Java

spring boot

Java

2 parte:

```
    public void setPassword(String password) {
        this.password = password;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}
```

spring boot

Java

Crear 2: 1 parte

UserController.java

```
package com.tutorialspoint;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.ui.ModelMap;

@Controller
public class UserController {

   @RequestMapping(value = "/user", method = RequestMethod.GET)
   public ModelAndView user() {
      return new ModelAndView("user", "command", new User());
   }
```

spring boot

Java

2:

```
    @RequestMapping(value = "/addUser", method = RequestMethod.POST)
    public String addUser(@ModelAttribute("SpringWeb")User user,
        ModelMap model) {
        model.addAttribute("username", user.getUsername());
        model.addAttribute("password", user.getPassword());
        model.addAttribute("address", user.getAddress());

        return "users";
    }
}
```

spring boot

Java

Vista 1 1:

user.jsp

```
<%@taglib uri = "http://www.springframework.org/tags/form" prefix = "form"%>
<html>
   <head>
      <title>Spring MVC Form Handling</title>
   </head>
   <body>

   <h2>User Information</h2>
   <form:form method = "POST" action = "/HelloWeb/addUser">
      <table>
         <tr>
            <td><form:label path = "username">User Name</form:label></td>
            <td><form:input path = "username" /></td>
         </tr>
         <tr>
            <td><form:label path = "password">Age</form:label></td>
            <td><form:password path = "password" /></td>
         </tr>
```

spring boot

Java

Vista 1:
2:

```
            <tr>
                <td><form:label path = "address">Address</form:label></td>
                <td><form:textarea path = "address" rows = "5" cols = "30" /></td>
            </tr>
            <tr>
                <td colspan = "2">
                    <input type = "submit" value = "Submit"/>
                </td>
            </tr>
        </table>
    </form:form>
    </body>
</html>
```

spring boot

Java

Vista 2:
1:

users.jsp

```
<%@taglib uri = "http://www.springframework.org/tags/form" prefix = "form"%>
<html>
   <head>
      <title>Spring MVC Form Handling</title>
   </head>
   <body>

      <h2>Submitted User Information</h2>
      <table>
         <tr>
            <td>Username</td>
            <td>${username}</td>
         </tr>
         <tr>
            <td>Password</td>
            <td>${password}</td>
         </tr>
         <tr>
```

spring boot

2:

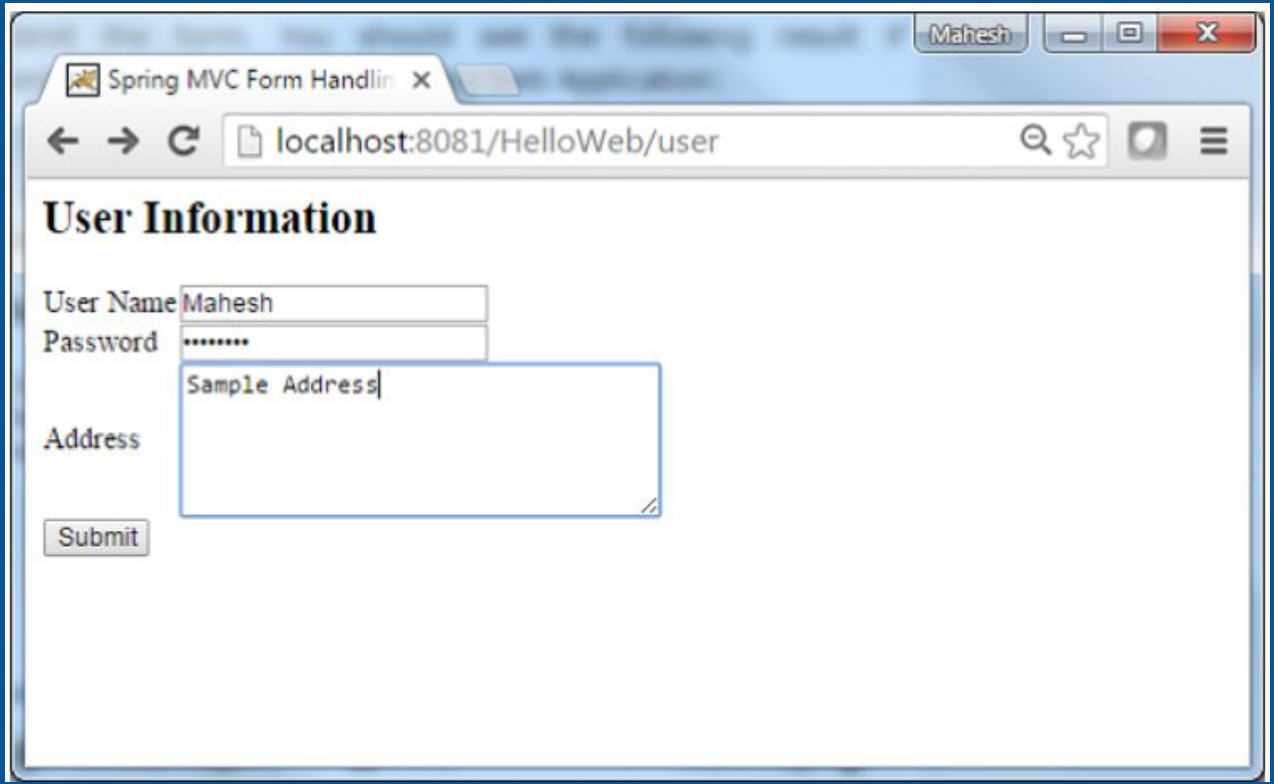
Java

```
        <tr>
            <td>Address</td>
            <td>${address}</td>
        </tr>
    </table>
  </body>
</html>
```

spring boot

Resultado:

Java



spring boot

Java

Spring MVC Form Handlin

localhost:8081/HelloWeb/addUser

Submitted User Information

Username Mahesh
Password Password
Address Sample Address